

Unity UGUI 基础

Building 项目实用版 · 从屏幕文字到世界空间说明牌

这不是通用控件百科，而是对 Assets/Scripts 中全部 UGUI 用法的整理。内容按“先摆放、再交互、后动态生成”的学习曲线编排，可直接对照项目脚本与 Prefab。

范围	本教程范围
保留	Canvas、CanvasScaler、RectTransform、Text、Image、RawImage、Button、GraphicRaycaster、EventSystem、Horizontal/Vertical Layout Group、LayoutElement
移除	原通用教程中与本项目无关的控件章节、示例需求和跨引擎对照
代码口径	项目使用 UnityEngine.UI.Text 与 LegacyRuntime.ttf；示例严格贴合现有代码

推荐学习顺序

- 1. Canvas 与缩放 → 2. RectTransform 锚点 → 3. Text / Image / RawImage
- 4. Button 与事件生命周期 → 5. 布局组 → 6. EventSystem 与射线
- 7. 小地图坐标换算 → 8. 运行时生成 UI → 9. World Space Canvas

版本说明：依据 2026-07-11 工作区脚本扫描编写。

1. 项目 UGUI 全景

项目的 UI 有两条主线：屏幕空间 UI 用于模式提示、菜单、准星、小地图和固定视角按钮；世界空间 UI 挂在场景物体上显示标题与说明。部分 UI 来自 Prefab，部分由脚本运行时创建。

难度	功能 / 脚本	实现原理与关键点
入门	模式文字 PlayerModeDisplay	Canvas + CanvasScaler + Text；左上角锚点；不接收射线
入门	详情准星 PlayerDetailInspectView	Canvas + 两个 Image；中心锚点；横竖条拼成十字
基础	设置面板 SettingsPanelController / Creator	Image、Button、Text、VerticalLayoutGroup、LayoutElement；Prefab
基础	预制体交换面板 PrefabSwapPanelView / Creator	底部提示、水平按钮排布、动态显示槽位
进阶	固定视角选择 PlayerFixedCameraView	运行时创建 Button / Image / Text；选中态；动态宽度
进阶	小地图传送 PlayerMinimapTeleportView	RawImage 显示 Texture；屏幕点转局部点；EventSystem
进阶	世界说明牌 WorldDescriptionUI	World Space Canvas；跟随、缩放、朝向相机、距离/模式显隐
补充	临时颜色提示 TemporaryColorHint	通过 Graphic 基类统一修改任意 UGUI 图形颜色

注意：PrefabSwapInteractable 自身不是 View，但它为按钮绑定闭包回调、控制面板显隐、鼠标状态和玩家输入锁，是 UI 与玩法连接的关键。

2. 第一块地基：Canvas 与 CanvasScaler

Screen Space - Overlay（项目主要模式）

模式提示、设置面板、预制体交换、固定视角、详情准星、小地图都覆盖在屏幕上。项目统一采用 1920 × 1080 参考分辨率、宽高匹配 0.5。

```
canvas.renderMode = RenderMode.ScreenSpaceOverlay;  
scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;  
scaler.referenceResolution = new Vector2(1920f, 1080f);  
scaler.matchWidthOrHeight = 0.5f;
```

Sorting Order: 谁盖在谁上面

UI	Sorting Order
模式提示	100
预制体交换面板	900
详情准星	960
小地图	980
设置面板	1000
世界说明牌	50 (World Space)

数值越大越靠前。这个顺序意味着设置面板会盖住其他屏幕 UI。新增全屏面板时，先查看现有区间，避免“明明激活却看不见”。

World Space（世界说明牌）

WorldDescriptionUI 把 Canvas 作为场景物体的子节点，尺寸用像素式单位定义，再用 0.01 的 Transform 缩放映射到世界尺寸。其位置每帧由 worldOffset 更新。

3. RectTransform: 摆放 UI 的核心

RectTransform 同时描述锚点、中心点、相对位置与尺寸。先决定“跟屏幕哪里走”，再调位置，最不容易乱。

名称	anchorMin / anchorMax	pivot	位置 / 尺寸
左上模式文字	(0,1) / (0,1)	(0,1)	(24,-24), 520×60
屏幕中心准星	(0.5,0.5) / 同	(0.5,0.5)	(0,0), 臂长决定尺寸
底部交互提示	(0.5,0) / 同	(0.5,0)	Y=110, 360×58
底部固定视角条	(0.5,0) / 同	(0.5,0)	Y=42, 宽度按按钮数计算
全屏铺满	(0,0) / (1,1)	默认	offsetMin/Max = 0
小地图内边距	(0,0) / (1,1)	默认	offsetMin=padding; offsetMax=-padding

三个最常用摆法

```
// 固定左上角
anchorMin = anchorMax = new Vector2(0f, 1f);
pivot = new Vector2(0f, 1f);

// 固定正中心
anchorMin = anchorMax = pivot = new Vector2(0.5f, 0.5f);

// 拉伸铺满父节点
anchorMin = Vector2.zero; anchorMax = Vector2.one;
offsetMin = offsetMax = Vector2.zero;
```

锚点相同时，sizeDelta 表示明确宽高；锚点分开时，offsetMin / offsetMax 表示与对应边的距离。

4. 最简单的显示组件

Text: 项目使用 Legacy UGUI Text

PlayerModeDisplay、按钮标签、固定视角文字、世界说明牌都使用 `UnityEngine.UI.Text`。项目字体来自 `Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf")`。

- `alignment` 决定文字在矩形中的对齐；模式文字为 `UpperLeft`，按钮多为 `MiddleCenter`。
- `horizontalOverflow` / `verticalOverflow` 决定换行与截断。世界说明正文使用 `Wrap + Truncate`。
- 纯显示文本设置 `raycastTarget = false`，避免挡住下面的按钮。

Image: Sprite 或纯色矩形

项目多数 `Image` 没有 `Sprite`，而是用 `color` 画半透明背景、按钮底色和准星。固定视角有图标时才给 `Image.sprite` 赋值，并打开 `preserveAspect`。

RawImage: 直接显示 Texture

小地图生成结果是 `Texture2D`，因此使用 `RawImage.texture`。`Image` 面向 `Sprite`，`RawImage` 面向 `Texture`；不要为了显示地图额外把 `Texture` 转成 `Sprite`。

```
minimapImage.texture = minimapTexture;  
minimapImage.color = Color.white;
```

Graphic: 共同基类

`TemporaryColorHint` 用 `Graphic` 同时覆盖 `Text`、`Image`、`RawImage` 等 UGUI 图形：缓存 `graphic.color`，临时变色，再插值恢复。这是“面向公共能力编程”的实用例子。

5. 入门练习：模式文字与详情准星

模式文字 PlayerModeDisplay

最小层级如下。脚本若未绑定 modeText，会在 Awake / Initialize 时自动创建。

```
Player Mode Canvas [Canvas, CanvasScaler]
└─ Current Mode Text [RectTransform, CanvasRenderer, Text]
```

- Canvas: Overlay, Sorting Order 100。
- Text: 左上锚点、左上 pivot、偏移 (24,-24)、大小 520×60。
- Refresh(viewMode, detailInspectActive) 只替换 text，不重建 UI。

详情准星 PlayerDetailInspectView

```
Detail Inspect Canvas
└─ Crosshair
    └─ Horizontal [Image, 36×2 默认]
    └─ Vertical [Image, 2×36 默认]
```

两条 Image 都锚在中心、位置为零。准星只用于视觉，raycastTarget=false。进入详情模式显示 Canvas；退出时隐藏并清除当前 WorldDescriptionUI 高亮。

练习：先在空场景里手工搭出这两种层级，再与脚本自动生成结果比较 RectTransform 数值。

6. Button：从摆放到事件生命周期

Button 依赖一个 Graphic 作为 targetGraphic。项目创建按钮时先加 Image，再加 Button，然后把 targetGraphic 指向 Image；子 Text 只显示标签。

```
Image image = buttonObject.AddComponent<Image>();
Button button = buttonObject.AddComponent<Button>();
button.targetGraphic = image;
button.onClick.AddListener(ClosePanel);
```

设置面板的绑定规则

SettingsPanelController 在 Awake 绑定“回到出生点”和“退出”，在 OnDestroy 解绑。绑定前先 RemoveListener，可避免某些重复初始化造成重复响应。

```
private void BindButtons() {
    closeButton.onClick.RemoveListener(ClosePanel);
    closeButton.onClick.AddListener(ClosePanel);
}
private void OnDestroy() { closeButton.onClick.RemoveListener(ClosePanel); }
```

动态列表必须捕获索引

PrefabSwapInteractable 和 PlayerFixedCameraView 都在循环中为按钮绑定不同索引。先复制到 capturedIndex，再创建回调；交换面板还保存 UnityAction，以便精确解绑。

```
int capturedIndex = i;
UnityAction action = () => SelectPrefab(capturedIndex);
button.onClick.AddListener(action);
```

UI 打开时还要同步玩家状态：解锁鼠标、显示鼠标、锁住玩家输入；关闭时恢复。设置面板可选把 Time.timeScale 设为 0，并保存旧值。

7. 布局组：让按钮自己排队

设置面板：VerticalLayoutGroup

```
Settings Canvas
└─ Settings Panel (全屏半透明 Image)
    └─ Content (520×280, Image, VerticalLayoutGroup)
        └─ Return Spawn Button (LayoutElement 高 72)
            └─ Close Button (LayoutElement 高 72)
```

- Padding: 左右 50, 上下 42; Spacing: 28。
- childControlWidth=true, childForceExpandWidth=true: 按钮吃满内容区可用宽度。
- childControlHeight=false: 高度由 LayoutElement.preferredHeight=72 决定。

交换面板：HorizontalLayoutGroup

中间 Content 为 1600×72, 最多 6 个选项按钮加 1 个退出按钮; 每个按钮 LayoutElement 为 190×64, 间距 18。不可用槽位直接 SetActive(false)。

固定视角：横排 + 手工算容器尺寸

按钮 84×84, 间距 16, 左右 padding 20。脚本按按钮数量计算面板宽度：

```
width = count * buttonSize.x
        + max(0, count - 1) * buttonSpacing + 40;
height = buttonSize.y + 24;
```

布局组负责“子物体怎么排”，RectTransform 负责“整组放在哪里”。二者职责不要混在一起。

8. GraphicRaycaster 与 EventSystem

可点击 UI 需要一条完整链路：Graphic 允许射线 → Canvas 上有 GraphicRaycaster → 场景有 EventSystem → EventSystem 上有与输入系统匹配的 Input Module。

名称	用途	使用场景
GraphicRaycaster	从屏幕指针检测 Canvas 中的 Graphic	设置、交换、小地图、固定视角 Canvas
EventSystem	维护当前选择并分发指针/导航事件	设置 Prefab 或运行时自动创建
StandaloneInputModule	旧输入系统适配	未启用新 Input System 时
InputSystemUIInputModule	新输入系统适配	ENABLE_INPUT_SYSTEM 时自动添加

PlayerFixedCameraView 与 PlayerMinimapTeleportView 会检查 EventSystem。启用新输入系统时添加 InputSystemUIInputModule，并禁用旧的 StandaloneInputModule，避免双模块争抢。

raycastTarget 的取舍

- 按钮背景 Image：保持开启，因为 Button 需要它命中。
- 按钮子 Text、准星 Image、纯显示文字：关闭，避免拦截。
- 全屏背景 Image 如果无需拦截，可以关闭；若要阻止点击穿透，则保留。

项目注意点：SettingsPanelPrefabCreator 把 EventSystem 与 Canvas 放在同一个 Prefab 根上。场景若已有 EventSystem，应避免再实例化第二个；小地图和固定视角的 EnsureEventSystem 更适合作为统一兜底。

9. 进阶：小地图点击如何变成世界坐标

PlayerMinimapTeleportView 的 RawImage 拉伸铺满 Canvas，并留 mapPadding。点击处理不是 Button.onClick，而是在模式激活时读取鼠标按下，再做坐标变换。

步骤	代码
1. 屏幕点 → UI 局部点	RectTransformUtility.ScreenPointToLocalPointInRectangle
2. 局部点 → 0~1	InverseLerp(rect.xMin,xMax,local.x), Y 同理
3. 0~1 → 世界 XZ	center + (normalized - 0.5) × worldSize
4. 贴地	从上方向下 Physics.Raycast, 命中点加 groundOffset

```
RectTransformUtility.ScreenPointToLocalPointInRectangle(  
+   minimapRect, pointerPosition, null, out localPoint);  
+float nx = Mathf.InverseLerp(rect.xMin, rect.xMax, localPoint.x);  
+float worldX = mapWorldCenter.x + (nx - 0.5f) * mapWorldSize.x;
```

Overlay Canvas 的事件相机参数传 null。若未来改成 Screen Space - Camera，应传对应 UI Camera，否则坐标会偏。还要在归一化后检查是否落在 0~1 内。

RawImage 的摆放

anchorMin=(0,0), anchorMax=(1,1), offsetMin=mapPadding, offsetMax=-mapPadding。负的 offsetMax 表示从右上边向内缩。

10. 进阶：运行时动态生成 UI

本项目大量采用“引用存在就复用，不存在就创建”的 Ensure 模式。它降低场景手工配置门槛，但必须保证重复调用不会重复创建。

```
if (selectionCanvas == null) { 创建 Canvas; }  
if (buttonContainer == null) { 创建面板与布局组; }  
RebuildButtons();
```

固定视角按钮 PlayerFixedCameraView

- 每个 cameraPoint 创建 RectTransform + Image + Button。
- 有 Sprite 时创建 Icon，并将锚点设为 0.12~0.88，开启 preserveAspect。
- 无 Sprite 时创建 Text，并启用 resizeTextForBestFit，范围 12~22。
- 点位不可用时 button.interactable=false；选中按钮改为 selectedButtonColor。

编辑器 Prefab 生成器

SettingsPanelPrefabCreator 与 PrefabSwapPanelPrefabCreator 在“工具/...”和“测试/...”菜单创建固定 Prefab，并用 SerializedObject 回填私有序列化引用。输出分别是：

- Assets/Prefab/UI/Settings Panel.prefab
- Assets/Prefab/UI/Prefab Swap Panel.prefab

Prefab 适合稳定、可视化调样式的面板；运行时生成适合数量和内容依赖数据的 UI。项目同时使用两种方式。

11. 难点：World Space 世界说明牌

```
场景物体 [WorldDescriptionUI]
├─ World Description Canvas [Canvas, GraphicRaycaster]
│   ├── Background [Image, 拉伸铺满]
│   ├── Title [Text, 上方 32%]
│   └─ Description [Text, 下方 68%]
```

摆放与分区

- Canvas sizeDelta 默认 320×160，localScale 默认 0.01。
- 标题 anchorMin=(0,0.68)，anchorMax=(1,1)，左右内缩 18。
- 正文 anchorMin=(0,0)，anchorMax=(1,0.68)，左右内缩 22。
- 背景铺满；标题和正文通过锚点比例切分区域。

每帧行为

- 位置 = transform.TransformPoint(worldOffset)，因此偏移跟随宿主朝向。
- 可选始终朝向 Camera.main；rotateOnlyYAxis 可限制为只水平旋转。
- 可按 maxVisibleDistance 显隐，或仅在详情查看射线命中时显示。

面向相机的方向

脚本用 canvas.position - camera.position 作为 LookRotation

方向，使画布正面按当前模型朝向约定对准相机。若换了

Canvas/文字资源后出现背面朝向，应先检查资源正面方向，再决定是否反转向量。

世界 UI 的可读性由

panelSize、canvasScale、字体大小和观看距离共同决定。先固定观看距离，再调缩放，最后调字号。

12. 面板状态与玩法状态要一起管理

玩法	UI 控制	UI 状态
设置面板	panelRoot 激活；玩家输入锁定；鼠标解锁；可选暂停	隐藏；输入解锁；恢复旧 timeScale
预制体交换	隐藏交互提示；锁输入；保存并切换预览相机； 显示鼠标	隐藏面板；恢复相机；根据范围恢复提示
固定视角选择	重建按钮；保证 EventSystem；显示鼠标	隐藏 Canvas；视项目流程恢复鼠标
详情查看	显示准星；中心射线寻找说明牌	隐藏准星；取消当前说明牌高亮
小地图	显示 RawImage；允许点击传送	隐藏 Canvas；停止处理点击

显隐用 SetActive 的影响

项目大多用 `gameObject.SetActive`。它会停止该节点及子节点的 Update 和渲染，适合完整面板。注意：若控制脚本也挂在被关闭的 `panelRoot` 上，它将无法自己再打开；项目把控制器挂在 Canvas 根或外部对象上，`panelRoot` 只负责视觉区域。

初始化顺序

- Awake / Initialize：补齐引用和 UI，绑定事件。
- OnDisable：关闭仍打开的状态，避免玩家输入或暂停残留。
- OnDestroy：解绑按钮，释放闭包引用。

这是 UI 稳定性的核心：视觉状态、输入状态、鼠标状态、时间状态必须成对恢复。

13. 按难度完成四个练习

练习 A：左上角状态文字（入门）

- 创建 Overlay Canvas 和 Text；设置参考分辨率 1920×1080。
- 锚到左上角，pivot 左上，偏移 24,-24；关闭 raycastTarget。

练习 B：两按钮设置面板（基础）

- 全屏半透明背景下放 520×280 内容区；加 VerticalLayoutGroup。
- 按钮加 LayoutElement，高 72；在 Awake 绑定，在 OnDestroy 解绑。

练习 C：数据驱动的横排按钮（进阶）

- 从列表创建 Button；正确捕获索引；无数据时显示提示 Text。
- 按按钮数计算面板宽度；用 interactable 与颜色表达状态。

练习 D：世界说明牌（进阶）

- World Space Canvas 设为 320×160，缩放 0.01；分出标题和正文。
- LateUpdate 跟随目标并朝向相机；加入距离显隐。

验收清单

- 改变 Game 视图比例后，角落和底部 UI 仍在正确位置。
- 纯显示 Text / Image 不会挡住按钮。
- 反复打开关闭面板，不会重复触发按钮或遗留输入锁。
- 场景只有一个有效 EventSystem，输入模块与项目输入系统一致。
- 世界说明牌近距离清晰、远距离按规则隐藏。

附录 A：组件速查（仅本项目用到）

名称	主要用途	重要属性
Canvas	屏幕覆盖或世界说明牌	Render Mode、Sorting Order
CanvasScaler	统一多分辨率缩放	Scale With Screen Size、1920×1080、0.5
RectTransform	定位、尺寸、拉伸、内边距	anchor、pivot、offset / sizeDelta
Text	模式、按钮、标签、说明	font、alignment、overflow、raycastTarget
Image	背景、按钮、图标、准星	color、sprite、preserveAspect、raycastTarget
RawImage	显示小地图 Texture2D	texture、RectTransform 边距
Button	设置、交换、固定视角	targetGraphic、interactable、onClick
LayoutElement	约束按钮首选宽高	preferredWidth / Height
VerticalLayoutGroup	设置按钮纵排	padding、spacing、子尺寸控制
HorizontalLayoutGroup	交换/视角按钮横排	spacing、childForceExpand
GraphicRaycaster	让 Canvas 可做图形射线检测	是否真的需要交互
EventSystem + Module	把输入分发给 UI	唯一性、输入系统匹配
Graphic	临时颜色提示的统一入口	color 与恢复时机

附录 B：脚本索引与排错

脚本名称	功能
Assets/Scripts/Player/PlayerModeDisplay.cs	左上角模式 Text
Assets/Scripts/Player/PlayerDetailInspectView.cs	中心准星与详情检测
Assets/Scripts/UI/SettingsPanelController.cs	设置面板显隐、按钮、暂停/输入
Assets/Scripts/Editor/SettingsPanelPrefabCreator.cs	生成设置面板 Prefab
Assets/Scripts/Interaction/PrefabSwapPanelView.cs	交换面板引用与槽位显示
Assets/Scripts/Interaction/PrefabSwapInteractable.cs	按钮绑定、预览相机、玩法状态
Assets/Scripts/Editor/PrefabSwapPanelPrefabCreator.cs	生成交换面板 Prefab
Assets/Scripts/Player/PlayerFixedCameraView.cs	动态固定视角按钮
Assets/Scripts/Player/PlayerMinimapTeleportView.cs	RawImage、点击坐标换算、传送
Assets/Scripts/WorldUI/WorldDescriptionUI.cs	世界空间说明牌
Assets/Scripts/Interaction/TemporaryColorHint.cs	Renderer / Graphic 临时变色

快速排错

问题	解决方案
按钮点不到	EventSystem → Input Module → GraphicRaycaster → targetGraphic/raycastTarget → 上层遮挡
窗口缩放后跑位	anchor → pivot → anchoredPosition → CanvasScaler
文字挡按钮	Text.raycastTarget=false
面板看不见	activeSelf / activeInHierarchy → Sorting Order → Canvas Render Mode
动态按钮重复响应	是否重复 AddListener; 是否在销毁时 RemoveListener
世界 UI 太大/太小	Canvas Rect 尺寸 → Transform scale → 字号 → 相机距离
小地图点击偏移	Canvas 模式与事件相机参数 → RawImage Rect → padding → 世界中心/范围

结论：本项目 UGUI 的重点不是控件数量，而是三件事——正确的 RectTransform、完整的输入链路、UI 状态与玩法状态的成对管理。